LAB N°1 — MACHINE LEARNING

GL4 — INSAT | October 2026

1. Objectives of the Lab

This Lab introduces the basic workflow of supervised machine learning using Python and Scikit-learn. The Iris flower dataset is used throughout the lab so that students can focus on the machine learning process rather than on data collection.

- Load, inspect and prepare a dataset using Pandas and NumPy.
- Explore numerical variables and the distribution of the three Iris species.
- Separate input features X from the target variable y.
- Build and evaluate a K-Nearest Neighbors (KNN) classifier.
- Study the effect of the value of k on classification accuracy.
- Build and evaluate a Logistic Regression classifier.
- Compare the performance of the two classification approaches.

2. Required Environment

Use Python 3 with the following libraries: pandas, numpy, scipy, matplotlib and scikit-learn. The exercises may be completed in Jupyter Notebook, Google Colab, or another Python environment available in the laboratory.

Required dataset: iris.csv. The dataset should contain the four numerical measurements (SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm), an identifier column such as Id, and a target column named Species.

Part 1 — Exploring and Preparing the Iris Dataset

The Iris dataset contains 150 observations belonging to three species: Iris-setosa, Iris-versicolor and Iris-virginica. Each observation contains four numerical measurements. The purpose of this part is to understand the structure and quality of the data before applying a machine learning algorithm.

Exercise 1 — Loading and Inspecting the Dataset

1. Load iris.csv into a Pandas DataFrame.
2. Display the shape of the DataFrame, its column names and data types, and the first three rows.
3. Display the number of observations and identify the numerical and categorical columns.
4. Check whether the dataset contains missing values or NaN values. Report the result clearly.

Exercise 2 — Descriptive Statistics and Class Distribution

5. Compute the main descriptive statistics of the four numerical features, including count, mean, standard deviation, minimum, maximum and quartiles.
6. Determine the number of observations belonging to each Iris species.
7. Compute the descriptive statistics of the numerical features separately for each species.

8. Briefly comment on which features appear to provide useful information for distinguishing the species.

Exercise 3 — DataFrame and Numerical Data Manipulation

9. Create a copy of the DataFrame and remove the Id column if it is present.
10. Access the first four cells of the DataFrame using row and column labels.
11. Create a 5 × 5 identity matrix with NumPy and convert it into a SciPy CSR sparse matrix.
12. Display the dense matrix and the corresponding CSR representation.

Part 2 — K-Nearest Neighbors Classification

K-Nearest Neighbors (KNN) is a supervised learning algorithm that classifies an observation according to the classes of its nearest training observations. In this part, the four numerical measurements are used as input features and Species is the target.

Exercise 4 — Preparing Training and Test Data

13. Define X as the four numerical features: SepalLengthCm, SepalWidthCm, PetalLengthCm and PetalWidthCm.
14. Define y as the Species target variable.
15. Encode the target labels as follows: Iris-setosa = 0, Iris-versicolor = 1 and Iris-virginica = 2.
16. Use train_test_split from Scikit-learn to create a training set containing 80% of the observations and a test set containing 20%. For 150 observations, this corresponds to 120 training observations and 30 test observations.
17. Use a fixed random_state so that the experiment is reproducible. Display the dimensions of X_train, X_test, y_train and y_test.

Exercise 5 — Building and Evaluating a KNN Classifier

18. Create a KNeighborsClassifier with k = 5.
19. Train the classifier using the training data.
20. Predict the species of the test observations.
21. Compute the classification accuracy on the test set.
22. Display a confusion matrix and briefly interpret the classification results.

Exercise 6 — Studying the Effect of k

23. Train and evaluate KNN classifiers for k values from 1 to 20 using the same training and test sets.
24. Store the test accuracy obtained for each value of k.
25. Create a plot with k on the horizontal axis and test accuracy on the vertical axis.
26. Identify the value of k that gives the highest test accuracy and report the corresponding accuracy.
27. Briefly explain why changing k can affect the performance of KNN.

Part 3 — Logistic Regression Classification

Logistic Regression is a supervised classification method that can be used to predict the class of an observation from its numerical features. The same Iris dataset and the same general train/test methodology are used so that the results can be compared with KNN.

Exercise 7 — Visual Exploration

28. Create a scatter plot using SepalLengthCm on the x-axis and PetalWidthCm on the y-axis.
29. Represent the three Iris species using different labels or visual markers.
30. Based on the plot, discuss whether these two features appear sufficient to clearly separate the three species.

Exercise 8 — Logistic Regression Model

31. Using the same X and y definitions as in the KNN part, split the dataset into 80% training data and 20% test data with a fixed random_state.
32. Create and train a LogisticRegression classifier using the training data.
33. Predict the species of the test observations.
34. Compute the test accuracy and display the confusion matrix.
35. Report the accuracy obtained by Logistic Regression.

Part 4 — Comparing the Two Classifiers

The final activity consolidates the results obtained with KNN and Logistic Regression. The objective is not simply to obtain a numerical accuracy, but to interpret the behavior of the two models on the same classification problem.

Exercise 9 — Performance Comparison

36. Create a small comparison table containing the test accuracy of KNN with k = 5, the best KNN model found in Exercise 6, and Logistic Regression.
37. Identify which model provides the highest test accuracy for the chosen train/test split.
38. Compare the confusion matrices of the best KNN model and Logistic Regression and identify which Iris species is the easiest or most difficult to classify.
39. Write a short conclusion explaining what you learned about data preparation, KNN, the choice of k, and Logistic Regression.

3. Expected Deliverables

Submit one Jupyter Notebook (.ipynb) containing the Python code, the main outputs, the requested plots and short interpretations. The notebook should be organized according to the exercises above. Every result should be accompanied by enough explanation to make the reasoning understandable.

4. Important Methodological Requirements

- Use a fixed random_state for train/test splitting so that results are reproducible.
- Do not use the Id column as a machine learning feature.
- Use only the training data to train the models; the test set is reserved for final evaluation.
- Use the same train/test split when comparing KNN and Logistic Regression.
- Clearly distinguish between X (input features) and y (target labels).
- Do not report only an accuracy value: include the requested plots, confusion matrices and short interpretations.